

```
#
```

```
# Minimalist mAI DSL — Resource-Parametric Functions
```

```
## The Language
```

```
### Resource declaration
```

```
...
```

```
resource Sonnet = anthropic("claude-sonnet-4-6")
```

```
resource Haiku  = anthropic("claude-haiku-4-5")
```

```
resource GPT    = openai("gpt-4o-mini", temperature=0.3)
```

```
...
```

```
### Type declaration
```

```
...
```

```
type Verdict { passed: Bool, feedback: Text, score: Int }
```

```
...
```

```
### Function declaration (resource-parametric)
```

Functions declare `on r` — the resource is bound at the call site, not at declaration.

The body IS the prompt. Return type IS the codegen directive.

Important : Note how prompt uses only named parameters/return type in the function signature — that's useful simplification from approach 2 that should make it both compatible with code generation and verifying inside prompt the variables (holes)

```
...
```

```
func write_code(task: Text) -> Code
```

```
  on r
```

```
  """Write clean Python for: {task}"""
```

```
func review(code: Code) -> Verdict
```

```
  on r
```

```
  """Review this code: {code}
```

```
  Return JSON: passed (bool), feedback (string), score (int)."""
```

```
func improve(code: Code, feedback: Text) -> Code
```

```
  on r
```

```
  """Improve code based on feedback. Code: {code} Feedback: {feedback}"""
```

```
...
```

```
### Builtins (no resource needed)
```

```
...
```

```
func read_pdf(path: File) -> Text  builtin
```

```
func read_text(path: File) -> Text builtin
```

```
func write_file(path: Text, content: Text) builtin
```

```
\ \ \
```

```
### Workflow — call site binds the resource
```

```
\ \ \
```

```
workflow main(task: Text):
```

```
    code    = write_code(task) on Sonnet
```

```
    verdict = review(code) on Haiku      // cheap model
```

```
    verdict2 = review(code) on GPT      // same func, different model
```

```
    improved = improve(code, verdict.feedback) on Sonnet
```

```
    write_file("output.py", improved)
```

```
\ \ \
```

```
---
```

```
## Complete Examples
```

```
### Calculator
```

```
\ \ \
```

```
resource Claude = anthropic("claude-sonnet-4-6")
```

```
func calculate(question: Text) -> Int
```

```
    on r
```

```
    """Compute the answer to: {question}. Reply with a single integer."""
```

```
workflow main(input: Text):
```

```
    result = calculate(input) on Claude
```

```
    print(result * 1000)
```

```
\ \ \
```

```
### Code Review Pipeline
```

```
\ \ \
```

```
resource Sonnet = anthropic("claude-sonnet-4-6")
```

```
resource Haiku  = anthropic("claude-haiku-4-5")
```

```
func write_code(task: Text) -> Code
```

```
    on r
```

```
    """Write Python for: {task}. Only code, no explanations."""
```

```
func review_code(code: Code) -> Text
```

```
    on r
```

```
    """Review this code and provide feedback: {code}"""
```

```
func improve_code(code: Code, feedback: Text) -> Code
```

```
    on r
```

```
    """Improve based on feedback. Code: {code} Feedback: {feedback}"""
```

```

workflow main(task: Text):
  code    = write_code(task) on Sonnet
  feedback = review_code(code) on Haiku
  improved = improve_code(code, feedback) on Sonnet
  write_file("improved.py", improved)
  ...

```

```

### PDF Summarizer with Structured Output
...

```

```

resource GPT = openai("gpt-4o-mini")

```

```

type Summary { title: Text, bullets: Text, word_count: Int }

```

```

func summarize(text: Text) -> Summary
  on r
  """Summarize concisely. {text}
  Return JSON: title (string), bullets (string), word_count (int)."""

```

```

workflow main(pdf: File):
  text    = read_pdf(pdf)
  summary = summarize(text) on GPT
  print(summary.title)
  ...

```

```

### Model Comparison (resource-parametric payoff)
...

```

```

resource Sonnet = anthropic("claude-sonnet-4-6")
resource GPT    = openai("gpt-4o")
resource Haiku  = anthropic("claude-haiku-4-5")

```

```

type Verdict { passed: Bool, feedback: Text, score: Int }

```

```

func review(code: Code) -> Verdict
  on r
  """Review this code: {code}. Return JSON: passed, feedback, score."""

```

```

workflow main(code_path: File):
  code = read_text(code_path)
  v1 = review(code) on Sonnet
  v2 = review(code) on GPT
  v3 = review(code) on Haiku
  print(v1.score)
  print(v2.score)
  print(v3.score)

```

```

---

## ## Type Checking (6 rules)

### ### Environments

```

$\Gamma : \text{name} \rightarrow \text{typ}$ (variables)
 $\Sigma : \text{name} \rightarrow (\text{typ list} \times \text{typ} \times \text{bool})$ (functions: params, return, needs_resource)
 $\Delta : \text{name} \rightarrow (\text{name} \times \text{typ}) \text{ list}$ (type decls: field lists)
 $R : \text{name set}$ (declared resources)

```

### ### Rules

#### \*\*1. Call with resource:\*\*

```

$\Sigma(f) = ([T_1, \dots, T_n], \text{Ret}, \text{true}) \quad \Gamma \vdash e_i : T_i \quad r \in R$

$\Gamma \vdash f(e_1, \dots, e_n) \text{ on } r : \text{Ret}$

```

#### \*\*2. Call without resource (builtin):\*\*

```

$\Sigma(f) = ([T_1, \dots, T_n], \text{Ret}, \text{false}) \quad \Gamma \vdash e_i : T_i$

$\Gamma \vdash f(e_1, \dots, e_n) : \text{Ret}$

```

#### \*\*3. Field access:\*\*

```

$\Gamma \vdash e : \text{TNamed}(S) \quad (f, T) \in \Delta(S)$

$\Gamma \vdash e.f : T$

```

#### \*\*4. Let binding:\*\*

```

$\Gamma \vdash e : T$

$\Gamma[x \mapsto T] \vdash \text{rest}$

```

**\*\*5. Arithmetic:\*\***  
`...`

$\Gamma \vdash e_1 : \text{TInt} \quad \Gamma \vdash e_2 : \text{TInt}$

---

$\Gamma \vdash e_1 \text{ op } e_2 : \text{TInt}$   
`...`

**\*\*6. Template validation (well-formedness):\*\***

It can be probably be even stronger as the rule below taking types into account  
`...`

fn has params  $(p_1, \dots, p_n)$  and prompt holes  $\{h_1, \dots, h_m\}$

---

$\{h_1, \dots, h_m\} = \{p_1, \dots, p_n\}$   
`...`

### Subtyping: `Code <: Text` (the only rule)

---

## Codegen: Return Type  $\rightarrow$  Output Handler

```
Return type	Generated Python
`Text`	`return _raw`
`Int`	`return int(_raw.strip())`
`Float`	`return float(_raw.strip())`
`Bool`	`return _raw.strip().lower() in ("true","yes","1")`
`Code`	strip markdown fences, `return _raw`
`TNamed(S)`	`return S.model_validate_json(_raw)`
```

### Agent  $\rightarrow$  SDK client

```
```python
# resource Sonnet = anthropic("claude-sonnet-4-6")  $\rightarrow$ 
import anthropic
_client_Sonnet = anthropic.Anthropic()
_model_Sonnet = "claude-sonnet-4-6"
...`
```

Function $\rightarrow$ Python (resource passed as argument)

```
```python
func review(code: Code) -> Verdict on r ""..."" $\rightarrow$
def review(code: str, *, _client, _model) -> Verdict:
```

```

_resp = _client.messages.create(
 model=_model, max_tokens=4096,
 messages=[{"role":"user","content":f"Review this code: {code}..."}]
)
_raw = _resp.content[0].text.strip()
_raw = _raw.replace("`json","").replace("`","").strip()
return Verdict.model_validate_json(_raw)
``,``,`

```

### Call site → binds client+model

```

``,`python
verdict = review(code) on Haiku →
verdict = review(code, _client=_client_Haiku, _model=_model_Haiku)
``,``,`

```

This is how resource-parametric functions compile: the function takes `\_client` and `\_model` as keyword args, and the call site fills them from the resource declaration.